

PID

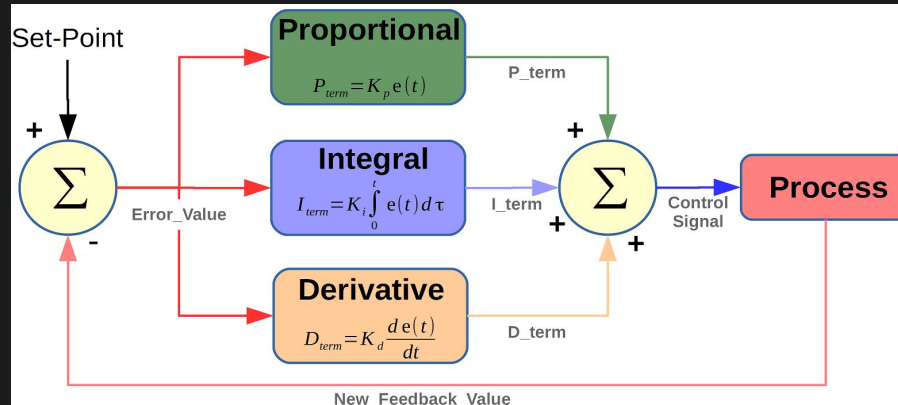
Micromouse @ UCI



**APPLY FOR
BOARD ! (MM
STAFF?????)**

What is PID?

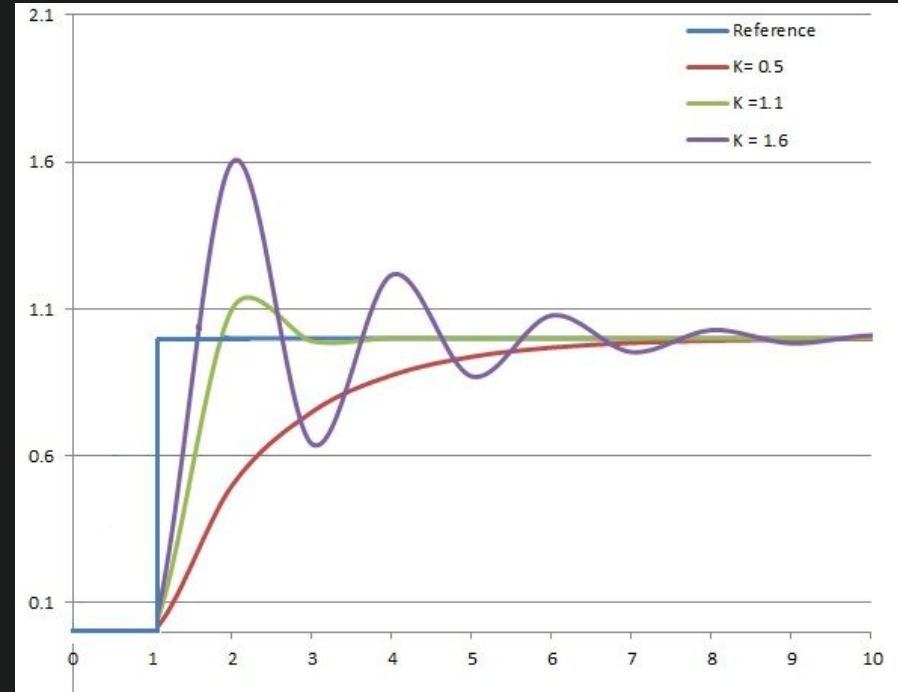
- Control algorithm
- “Proportional-Integral-Derivative” control
- Used to direct actuators to do a certain action
- I.e. control speed, position, angle, etc
- Can be layered



**How do you come
to a stop at a stop
sign?**

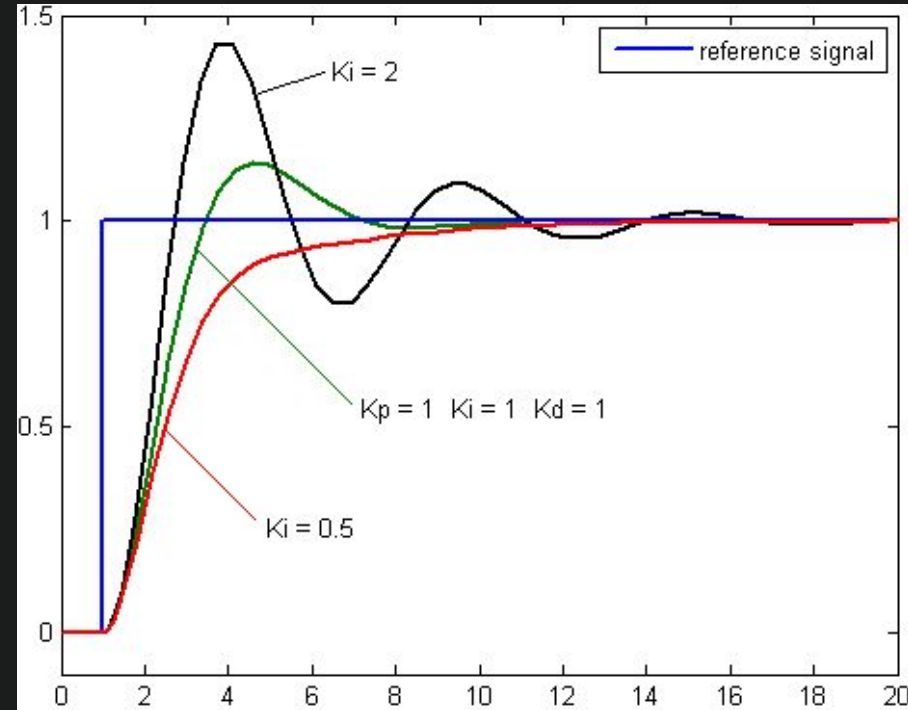
Proportional

- Proportional to the error term $e(t)$
- K = proportional gain
 - Manipulation of this changes the response time of control system
- K too large $\rightarrow$ overshooting reference
- K too small $\rightarrow$ too slow of a response



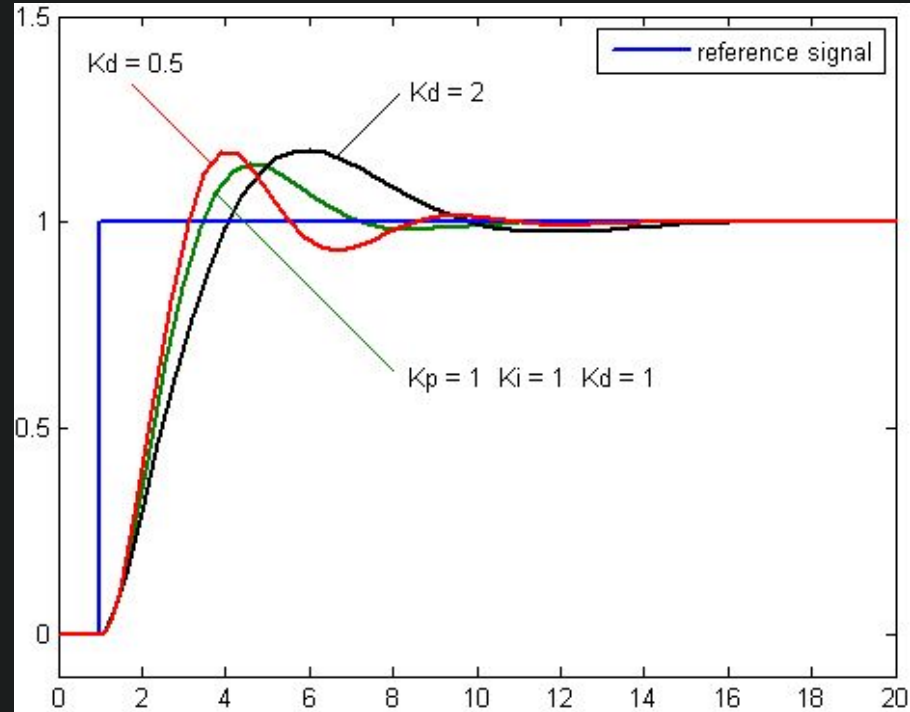
Integral

- Addresses the steady state error
 - Difference between desired and actual output
- Takes into account error over time
- Adjusts based on changes
 - Integral term increases if error is positive and decreases if error is negative
- Compensates for **“steady state error”**



Derivative

- Proportional to rate of change of proportional variable
 - Responsive to rapid changes
- Helps predict future error
- Improves stability and settling of output to desired output
- Dampening



Tuning

- Good PID needs tuning of coefficients K for each term
- We will be tuning manually
- There are algorithmic ways to tune to get optimal coefficients
 - Out of scope for MM

PID Code

- Integrating & tuning PID with the motors is your challenge
 - Hint: start with P, then go to D, do I last, if it's even necessary

```
float calculate(float target, float current) {  
    float error = target - current;  
  
    _currentTime = HAL_GetTick();  
    float deltaTime = _currentTime - _lastTime;  
  
    integratedError += error / deltaTime;  
  
    float p = kP * error;  
    float i = kI * integratedError;  
    float d = kD * ((error - lastError) / deltaTime);  
  
    lastError = error;  
  
    _lastTime = _currentTime;  
  
    return p + d + v + s;  
}
```

STM32 Debugger Tutorial!